

UNIVERSIDAD ARGENTINA DE LA EMPRESA

Programación Orientada a Objetos

TRABAJO PRÁCTICO OBLIGATORIO

Propuesta C: Sistema de movilidad compartida (bicicletas y monopatines)

Etapa 1 Modelado y aplicación de patrones

Integrantes:
Valentino Cinti - Legajo: 1213639
Ian Holm - Legajo: 1214983
Lautaro Piston - Legajo: 1215780

Docente: Leon Maria Angela
Curso: Miércoles Turno tarde

Índice

1. Análisis del enunciado.....2
1.1 Actores.....2
1.2 Requerimientos funcionales.....2
Usuarios y cuentas.....2
Viajes.....2
Tarifas y promociones.....2
Vehículos y mantenimiento.....3
Estaciones y redistribución.....3
Personal y reportes.....3
1.3 Requerimientos no funcionales.....3
1.4 Reglas de negocio.....3
Usuarios y saldo.....3
Vehículos.....3
Estaciones.....4
Viajes y tarifas.....4
1.5 Supuestos.....4
Usuarios y saldo.....4
Viajes y tarifas.....5
Vehículos y mantenimiento.....5
Estaciones.....5
Personal.....5
General.....6
2. Diagrama de casos de uso.....7
2.1 Diagrama - link a Draw.io.....7
2.2 Descripción de los casos de uso.....8
3. Diagrama de clases inicial.....17
3.1 Diagrama - link a Draw.io.....17
4. Diagramas de secuencia.....18
4.1 CU13 Reportar falla - Link a Draw.io.....18
4.2 CU15 Registrar traslado - Link a Draw.io.....19
5. Aplicación de patrones.....20
5.1 Principios SOLID.....20
Responsabilidad única (SRP).....20
Abierto/Cerrado (OCP).....20
Segregación de interfaces (ISP).....20
5.2 Patrones GRASP.....21
Experto en información.....21
Creador.....21
Controlador.....21
Polimorfismo.....21
6. Diagrama de clases final.....22
6.1 Diagrama - link a Draw.io.....22
6.2 Cambios respecto del diagrama inicial.....22

1. Análisis del enunciado

1.1 Actores

Usuario, operador del sistema, técnico de mantenimiento y administrador.

1.2 Requerimientos funcionales

Ampliación del alcance. Además de los requerimientos del enunciado, el grupo incorporó una funcionalidad que permite al administrador agregar nuevas estaciones definiendo los tipos de vehículos que admite cada una y su capacidad por tipo, previendo que el municipio amplíe el servicio en el futuro.

Usuarios y cuentas

RF01: Registrar usuarios con nombre, documento, correo electrónico y fecha de alta.

RF02: Asociar y quitar medios de pago (tarjeta o billetera virtual).

RF03: Recargar saldo con un medio de pago asociado.

RF04: Consultar el saldo y el historial de movimientos (recargas y débitos).

RF05: Suscribir y dar de baja el plan mensual.

Viajes

RF06: Consultar las estaciones y sus vehículos disponibles.

RF07: Retirar un vehículo de una estación e iniciar un viaje.

RF08: Cancelar un retiro.

RF09: Registrar los puntos del recorrido (coordenadas y hora) durante el viaje.

RF10: Devolver un vehículo en una estación y cerrar el viaje.

RF11: Calcular la tarifa del viaje aplicando la promoción más favorable y debitarla de la cuenta.

RF12: Registrar notas de ajuste sobre viajes cerrados (operador).

RF13: Detectar los vehículos no devueltos en 24 horas, marcarlos como perdidos y aplicar el cargo.

Tarifas y promociones

RF14: Registrar tarifas por tipo de vehículo con fecha de vigencia.

RF15: Registrar promociones con nombre, período de vigencia y condiciones.

Vehículos y mantenimiento

RF16: Dar de alta y de baja vehículos de los tres tipos.

RF17: Actualizar el estado y el nivel de batería de los vehículos.

RF18: Reportar fallas de un vehículo (usuario u operador).

RF19: Generar órdenes de mantenimiento asignadas a un técnico (operador).

RF20: Cerrar órdenes de mantenimiento registrando técnico, fecha, tareas y repuestos.

Estaciones y redistribución

RF21: Dar de alta y modificar estaciones, indicando los tipos de vehículos que admite cada una y la capacidad máxima para cada tipo, de modo que el administrador pueda incorporar nuevas estaciones si el municipio decide ampliar el servicio.

RF22: Generar alertas de redistribución según la ocupación.

RF23: Registrar traslados de vehículos entre estaciones (origen, destino, vehículos y operador).

Personal y reportes

RF24: Dar de alta al personal (operadores, técnicos y administrador).

RF25: Emitir el reporte de viajes por usuario.

RF26: Emitir el reporte de uso por estación.

RF27: Emitir el reporte de vehículos con más fallas.

1.3 Requerimientos no funcionales

RNF01: Desarrollado en Java 17 o superior, organizado en paquetes.

RNF02: Los datos persisten en archivos y se conservan entre ejecuciones.

RNF03: Interfaz gráfica desarrollada con Swing.

RNF04: Código versionado en GitHub.

RNF05: La interfaz se adapta al tamaño de la ventana.

1.4 Reglas de negocio

Usuarios y saldo

RN01: Un usuario solo puede retirar un vehículo si su saldo alcanza el mínimo establecido.

RN02: Un usuario no puede tener más de un viaje abierto a la vez.

RN03: Si al terminar un viaje el saldo queda negativo, el usuario queda inhabilitado hasta regularizarlo. Un usuario inhabilitado no puede retirar vehículos.

Vehículos

RN04: Existen tres tipos de vehículo: bicicleta común, bicicleta eléctrica y monopatín eléctrico.

RN05: Los estados posibles son disponible, en uso, en reparación, batería baja y fuera de servicio. Solo un vehículo disponible puede retirarse.

RN06: Un vehículo eléctrico con batería menor al 15 % pasa a batería baja y no se ofrece. Las bicicletas comunes no tienen batería.

RN07: Un vehículo con 3 reportes de falla en 30 días pasa a mantenimiento obligatorio.

RN08: Una orden de mantenimiento no se cierra sin registrar técnico, fecha y tareas realizadas.

Estaciones

RN09: Cada estación tiene una capacidad fija. No se puede devolver un vehículo en una estación llena ni fuera de una estación.

RN10: Si la estación de destino está llena, el usuario dispone de 10 minutos adicionales sin cargo para llegar a otra.

RN11: Una estación con menos del 20 % o más del 90 % de ocupación genera una alerta de redistribución.

Viajes y tarifas

RN12: La tarifa es un cargo de desbloqueo más un precio por minuto, según el tipo de vehículo: bicicleta común $200 de desbloqueo + $20 por minuto; bicicleta eléctrica $300 + $45 por minuto; monopatín eléctrico $300 + $50 por minuto.

RN13: Un retiro cancelado dentro de los 2 minutos no tiene cargo.

RN14: El tiempo máximo de un viaje es de 2 horas. Cada minuto que exceda ese límite se cobra con un recargo del 50 %.

RN15: Si el vehículo no se devuelve en 24 horas, se lo reporta como perdido y se aplica un cargo fijo.

RN16: Promoción de primer viaje: el primer viaje de cada usuario es gratis hasta 15 minutos.

RN17: Plan mensual: incluye 20 viajes de hasta 30 minutos.

RN18: Las promociones no se acumulan: se aplica la más favorable para el usuario.

RN19: Un viaje cerrado no se modifica. Las correcciones se hacen con una nota de ajuste del operador.

1.5 Supuestos

Usuarios y saldo

S01: El saldo mínimo para retirar un vehículo equivale al cargo de desbloqueo más 30 minutos de uso del tipo de vehículo a retirar: $800 para la bicicleta común, $1.650 para la bicicleta eléctrica y $1.800 para el monopatín.

S02: El importe del viaje se debita al cerrarlo, aunque el saldo quede negativo.

S03: Un usuario inhabilitado se rehabilita automáticamente cuando una recarga deja su saldo en cero o más.

S04: Los pagos son simulados: toda recarga realizada con un medio de pago asociado se considera aprobada.

S05: No se utilizan contraseñas. El usuario se identifica con su documento y el personal con su legajo.

Viajes y tarifas

S06: Toda fracción de minuto se cobra como un minuto completo.

S07: En el primer viaje se bonifican el cargo de desbloqueo y los primeros 15 minutos. Los minutos restantes se cobran a tarifa normal.

S08: El plan mensual cuesta $15.000 por mes calendario y se debita del saldo al suscribirse. Los viajes incluidos no pagan desbloqueo y tienen 30 minutos bonificados, y el excedente se cobra a tarifa normal. A partir del viaje 21 del mes se cobra la tarifa completa.

S09: El recargo del 50 % se aplica solo a los minutos que superan las 2 horas.

S10: El cargo fijo por vehículo perdido es de $150.000. El viaje queda en estado “perdido” y el vehículo pasa a “fuera de servicio”.

S11: Se aplica la tarifa vigente en la fecha y hora de inicio del viaje.

S12: Ante un retiro cancelado, el vehículo vuelve a la estación de origen en estado “disponible” y el viaje queda como “cancelado”.

S13: Los puntos del recorrido se ingresan o simulan. La distancia del viaje se calcula sumando las distancias entre puntos consecutivos.

Vehículos y mantenimiento

S14: Al llegar a 3 fallas en 30 días, el sistema pasa el vehículo a “en reparación” y avisa al operador, que genera la orden de mantenimiento.

S15: Reportar una falla durante un viaje no lo cierra: el usuario debe devolver el vehículo igualmente.

S16: El nivel de batería se informa al devolver el vehículo. Un vehículo con batería baja vuelve a “disponible” cuando un técnico registra la carga.

Estaciones

S17: Cuando el usuario intenta devolver en una estación llena, el sistema registra el intento y descuenta hasta 10 minutos del tiempo cobrado del viaje.

S18: La ocupación de una estación se calcula como la cantidad de vehículos presentes, en cualquier estado, dividida por su capacidad total. Se evalúa después de cada retiro, devolución o traslado.

S19: La capacidad total de una estación es la suma de sus capacidades por tipo de vehículo. Una estación se considera llena para un tipo cuando alcanza la capacidad definida para ese tipo.

Personal

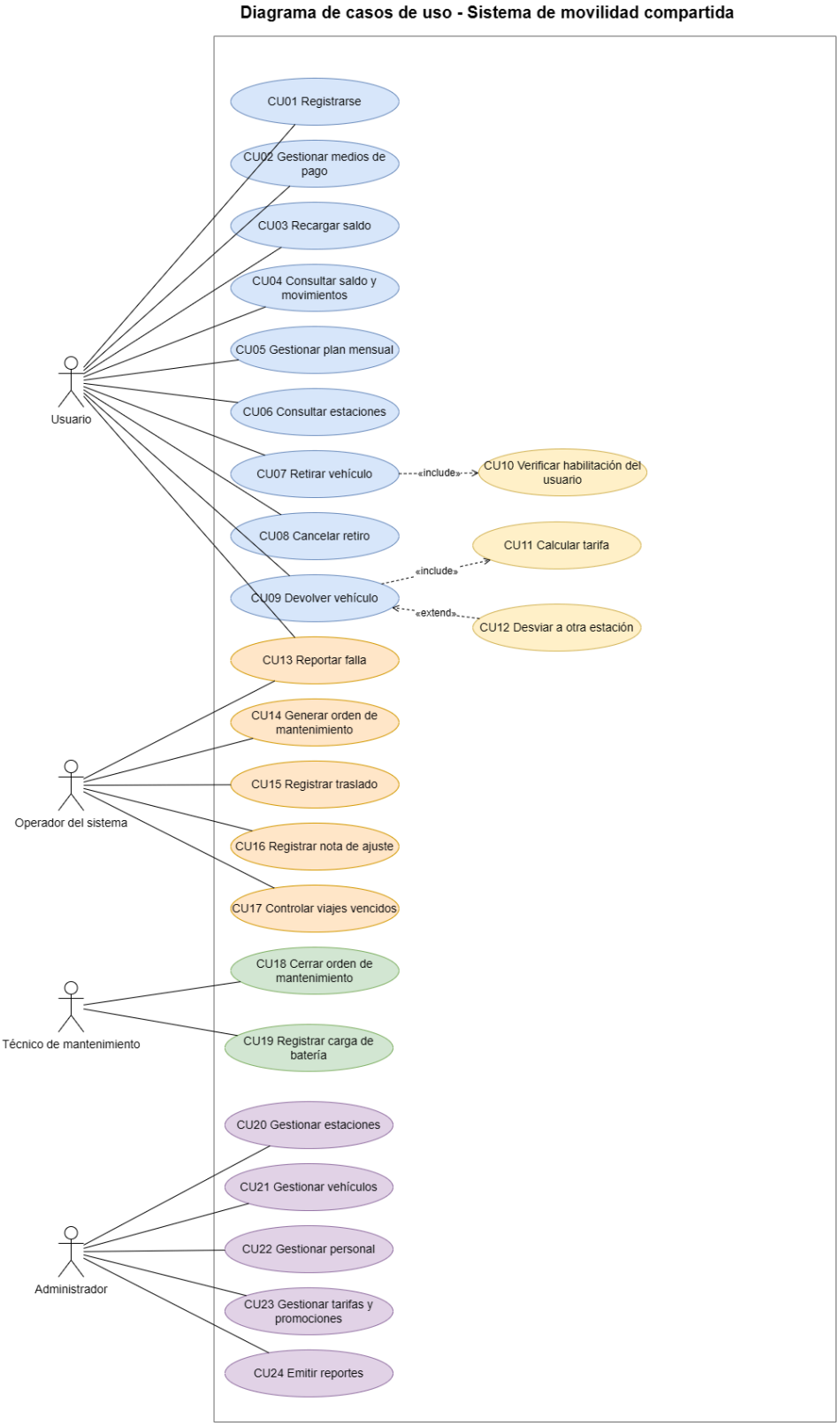
S20: Existe un único administrador.

General

S21: El sistema utiliza un reloj configurable, para poder probar las reglas que dependen del tiempo (2 horas, 24 horas, 30 días) sin esperar en tiempo real.

2. Diagrama de casos de uso

2.1 Diagrama - link a Draw.io



2.2 Descripción de los casos de uso

CU01: Registrarse	
Actor	Usuario
Precondición	El documento ingresado no está registrado en el sistema.
Flujo principal	1. El usuario ingresa nombre, documento y correo electrónico. 2. El sistema valida los datos. 3. El sistema crea el usuario con una cuenta con saldo cero y registra la fecha de alta.
Flujos alternativos	2a. Si el documento ya está registrado o falta algún dato, el sistema informa el error y no crea al usuario.

CU02: Gestionar medios de pago	
Actor	Usuario
Precondición	El usuario está registrado.
Flujo principal	1. El usuario elige agregar o quitar un medio de pago. 2. Para agregarlo, indica el tipo (tarjeta o billetera virtual) y su identificador. 3. El sistema guarda el cambio.
Flujos alternativos	2a. Si los datos están incompletos, el sistema rechaza el alta. 1a. Si el usuario quiere quitar un medio que no tiene asociado, el sistema informa el error.

CU03: Recargar saldo	
Actor	Usuario
Precondición	El usuario tiene al menos un medio de pago asociado.
Flujo principal	1. El usuario elige el medio de pago y el monto. 2. El sistema acredita el monto en la cuenta y registra un movimiento de recarga. 3. Si el usuario estaba inhabilitado y su saldo queda en cero o más, el sistema lo rehabilita (S03).

Flujos alternativos	1a. Si el monto es menor o igual a cero, el sistema lo rechaza. 1b. Si no tiene medios de pago, el sistema le pide asociar uno (CU02).

CU04: Consultar saldo y movimientos	
Actor	Usuario
Precondición	El usuario está registrado.
Flujo principal	1. El usuario solicita la consulta. 2. El sistema muestra el saldo actual y los movimientos ordenados por fecha, del más reciente al más antiguo.
Flujos alternativos	2a. Si no hay movimientos, el sistema muestra solo el saldo e informa que no hay movimientos.

CU05: Gestionar plan mensual	
Actor	Usuario
Precondición	El usuario está registrado.
Flujo principal	1. El usuario elige suscribirse al plan mensual. 2. El sistema verifica que el saldo alcance el costo del plan. 3. El sistema debita el costo (S08) y activa el plan para el mes en curso.
Flujos alternativos	2a. Si el saldo no alcanza, el sistema rechaza la suscripción. 1a. Si el usuario da de baja el plan, sigue vigente hasta fin de mes y no se renueva.

CU06: Consultar estaciones	
Actor	Usuario
Precondición	Ninguna.
Flujo principal	1. El usuario solicita ver las estaciones. 2. El sistema muestra cada estación con su dirección, su ocupación y la cantidad de vehículos disponibles por tipo.

Flujos alternativos	2a. Si una estación no tiene vehículos disponibles, se muestra igual indicando cero.

CU07: Retirar vehículo	
Actor	Usuario
Precondición	El usuario está registrado y se encuentra en una estación.
Flujo principal	1. El usuario elige la estación y el vehículo. 2. El sistema verifica que el vehículo esté disponible (en los eléctricos, con batería igual o mayor al 15 %). 3. El sistema verifica la habilitación del usuario. 4. El sistema saca el vehículo de la estación, lo pasa a "en uso" y crea el viaje con la hora de inicio. 5. El sistema evalúa la ocupación de la estación y, si queda por debajo del 20 %, genera una alerta de redistribución. 6. Mientras dura el viaje, el sistema registra los puntos del recorrido (coordenadas y hora), que se ingresan o simulan (S13).
Flujos alternativos	2a. Si el vehículo no está disponible, el sistema lo informa y no inicia el viaje. 3a. Si el usuario está inhabilitado, tiene un viaje abierto o no alcanza el saldo mínimo, el sistema informa el motivo y no inicia el viaje.

CU08: Cancelar retiro	
Actor	Usuario
Precondición	El usuario tiene un viaje en curso.
Flujo principal	1. El usuario solicita cancelar el retiro. 2. El sistema verifica que no pasaron más de 2 minutos desde el inicio. 3. El sistema devuelve el vehículo a la estación de origen como disponible y marca el viaje como cancelado, sin cargo (S12).
Flujos alternativos	2a. Si pasaron más de 2 minutos, el sistema no permite cancelar y el usuario debe devolver el vehículo (CU09).

CU09: Devolver vehículo	
Actor	Usuario

Precondición	El usuario tiene un viaje en curso.
Flujo principal	1. El usuario indica la estación de devolución y, si el vehículo es eléctrico, el nivel de batería. 2. El sistema verifica que la estación tenga lugar para ese tipo de vehículo. 3. El sistema cierra el viaje con la hora de fin y calcula la distancia recorrida a partir de los puntos del recorrido registrados (S13). 4. El sistema incorpora el vehículo a la estación como disponible, o como batería baja si es eléctrico y tiene menos del 15 %. 5. El sistema calcula el importe (CU11) y lo debita de la cuenta. 6. Si el saldo queda negativo, el sistema inhabilita al usuario. 7. El sistema evalúa la ocupación y, si supera el 90 %, genera una alerta de redistribución.
Flujos alternativos	2a. Si la estación está llena, se ejecuta CU12 y el viaje sigue abierto.

CU10: Verificar habilitación del usuario	
Actor	Incluido en CU07
Precondición	Hay un retiro en curso.
Flujo principal	1. El sistema verifica que el usuario no esté inhabilitado. 2. Verifica que no tenga otro viaje abierto. 3. Verifica que el saldo alcance el mínimo del tipo de vehículo (S01).
Flujos alternativos	1a, 2a, 3a. Si falla alguna verificación, el sistema corta el retiro con una excepción que indica el motivo.

CU11: Calcular tarifa	
Actor	Incluido en CU09
Precondición	El viaje está cerrado.
Flujo principal	1. El sistema obtiene la tarifa vigente al inicio del viaje para el tipo de vehículo (S11). 2. Calcula los minutos cobrables, redondeando hacia arriba (S06) y descontando hasta 10 minutos si hubo una estación llena (S17). 3. Calcula el cargo de desbloqueo más los minutos por el precio, con un recargo del 50 % sobre los minutos que superan las 2 horas (S09). 4. Evalúa las promociones que aplican y descuenta la más favorable para el usuario (RN18).

Flujos alternativos	4a. Si ninguna promoción aplica, se cobra la tarifa completa.

CU12: Desviar a otra estación	
Actor	Usuario
Precondición	La estación elegida para devolver está llena para ese tipo de vehículo.
Flujo principal	1. El sistema registra el intento de devolución y su hora. 2. El sistema informa que el usuario dispone de 10 minutos sin cargo y le muestra las estaciones cercanas con lugar. 3. El usuario devuelve el vehículo en otra estación (continúa en CU09).
Flujos alternativos	3a. Si tarda más de 10 minutos en llegar, los minutos excedentes se cobran a tarifa normal.

CU13: Reportar falla	
Actor	Usuario, operador del sistema
Precondición	El vehículo existe en el sistema.
Flujo principal	1. El actor indica el vehículo y una descripción de la falla. 2. El sistema registra el reporte con la fecha. 3. Si el vehículo acumula 3 fallas en los últimos 30 días, el sistema lo pasa a "en reparación" y avisa al operador (S14).
Flujos alternativos	1a. Si el código de vehículo no existe, el sistema informa el error.

CU14: Generar orden de mantenimiento	
Actor	Operador del sistema
Precondición	El vehículo tiene fallas reportadas.
Flujo principal	1. El operador selecciona el vehículo y el técnico. 2. El sistema crea la orden con la fecha de apertura y la asigna al técnico. 3. El vehículo queda "en reparación".
Flujos alternativos	1a. Si el vehículo ya tiene una orden abierta, el sistema lo informa y no crea otra.

CU15: Registrar traslado	
Actor	Operador del sistema
Precondición	Existe una alerta de redistribución o el operador decide mover vehículos.
Flujo principal	1. El operador consulta las alertas de redistribución. 2. Elige la estación de origen, la de destino y los vehículos a mover. 3. El sistema verifica que el destino tenga lugar. 4. El sistema mueve los vehículos y registra el traslado con la fecha y el operador responsable. 5. El sistema evalúa la ocupación de ambas estaciones y, si alguna queda fuera del rango, genera una alerta de redistribución (RN11).
Flujos alternativos	2a. Si un vehículo seleccionado está en uso, no se puede trasladar. 3a. Si el destino no tiene lugar suficiente, el sistema rechaza el traslado.

CU16: Registrar nota de ajuste	
Actor	Operador del sistema
Precondición	El viaje está cerrado.
Flujo principal	1. El operador selecciona el viaje e indica el motivo y el importe del ajuste. 2. El sistema registra la nota sin modificar el viaje y acredita o debita el importe en la cuenta del usuario (RN19).
Flujos alternativos	1a. Si el viaje no está cerrado, el sistema rechaza la nota.

CU17: Controlar viajes vencidos	
Actor	Operador del sistema
Precondición	Ninguna (el control también se ejecuta al iniciar el sistema).
Flujo principal	1. El operador solicita el control. 2. El sistema busca los viajes en curso con más de 24 horas. 3. Para cada uno, marca el viaje como perdido, pasa el vehículo a "fuera de servicio" y debita el cargo fijo (S10).

Flujos alternativos	2a. Si no hay viajes vencidos, el sistema lo informa.

CU18: Cerrar orden de mantenimiento	
Actor	Técnico de mantenimiento
Precondición	El técnico tiene una orden abierta asignada.
Flujo principal	1. El técnico selecciona la orden. 2. Ingresa la fecha de cierre, las tareas realizadas y los repuestos utilizados. 3. El sistema cierra la orden y el vehículo vuelve a "disponible".
Flujos alternativos	2a. Si falta la fecha o las tareas, el sistema no cierra la orden (RN08).

CU19: Registrar carga de batería	
Actor	Técnico de mantenimiento
Precondición	El vehículo es eléctrico y está en "batería baja".
Flujo principal	1. El técnico selecciona el vehículo e ingresa el nuevo nivel de batería. 2. El sistema actualiza el nivel y, si es igual o mayor al 15 %, pasa el vehículo a "disponible" (S16).
Flujos alternativos	1a. Si el vehículo no es eléctrico, el sistema rechaza la operación.

CU20: Gestionar estaciones	
Actor	Administrador
Precondición	Ninguna.
Flujo principal	1. El administrador elige dar de alta o modificar una estación. 2. Ingresa código, dirección, coordenadas, tipos de vehículo admitidos y capacidad por tipo. 3. El sistema guarda la estación.
Flujos alternativos	2a. Si el código ya existe o alguna capacidad es menor o igual a cero, el sistema lo rechaza. 2b. Si la nueva capacidad es menor que la cantidad de vehículos presentes, el sistema rechaza la modificación.

CU21: Gestionar vehículos	
Actor	Administrador
Precondición	Existe al menos una estación.
Flujo principal	1. Para el alta, el administrador indica el tipo, el código, los datos propios del tipo y la estación inicial. 2. El sistema registra el vehículo como disponible. 3. Para la baja, el sistema pasa el vehículo a "fuera de servicio".
Flujos alternativos	1a. Si la estación no tiene lugar para ese tipo, el sistema rechaza el alta. 3a. Si el vehículo está en uso, no puede darse de baja.

CU22: Gestionar personal	
Actor	Administrador
Precondición	Ninguna.
Flujo principal	1. El administrador da de alta un operador o un técnico con legajo, nombre y teléfono. 2. El sistema lo registra.
Flujos alternativos	1a. Si el legajo ya existe, el sistema rechaza el alta.

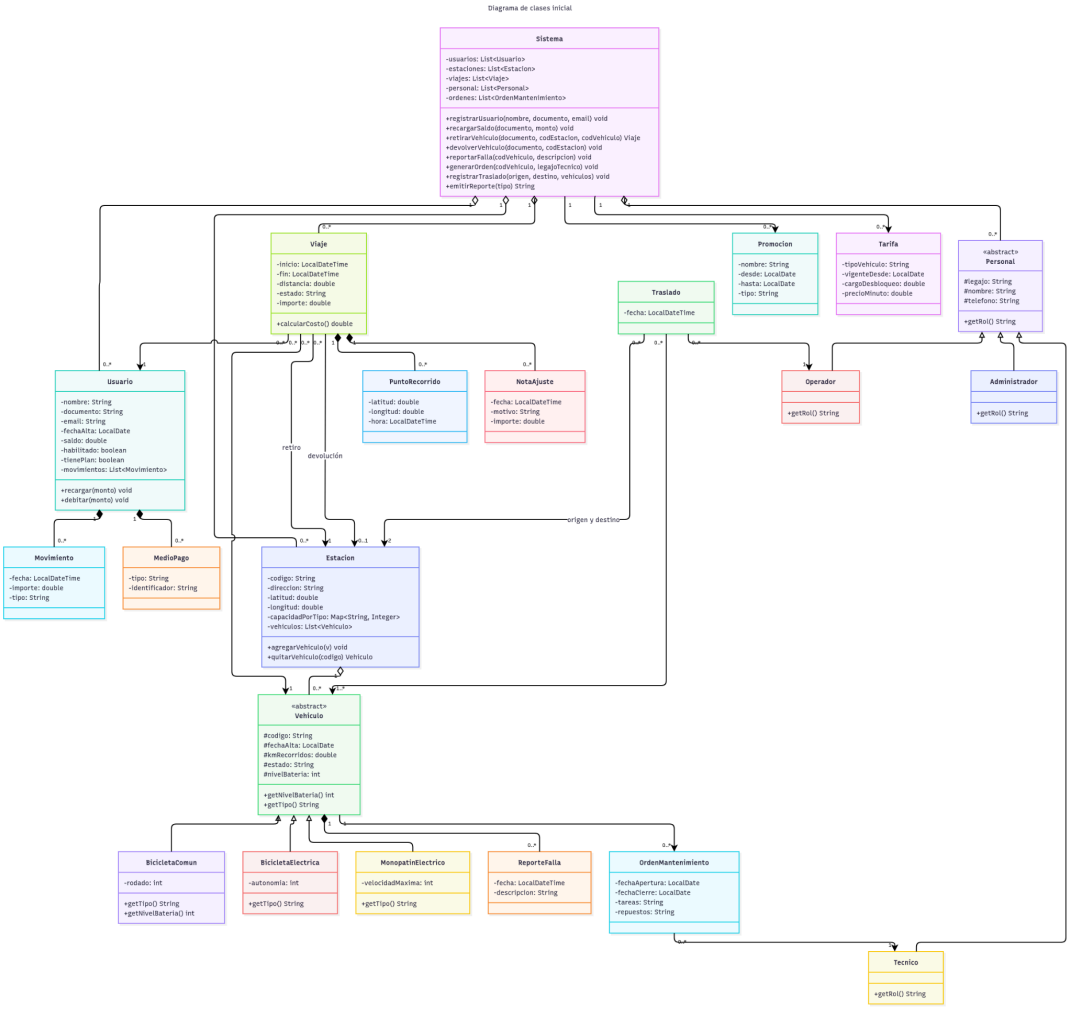
CU23: Gestionar tarifas y promociones	
Actor	Administrador
Precondición	Ninguna.
Flujo principal	1. El administrador carga una nueva tarifa para un tipo de vehículo, con su fecha de vigencia, sin modificar las anteriores. 2. También puede dar de alta promociones con nombre, período de vigencia y condiciones. 3. El sistema las guarda.
Flujos alternativos	1a. Si algún valor es negativo o la fecha de vigencia es inválida, el sistema lo rechaza.

CU24: Emitir reportes

Actor	Administrador
Precondición	Ninguna.
Flujo principal	1. El administrador elige el reporte: viajes por usuario, uso por estación (retiros y devoluciones) o vehículos con más fallas. 2. El sistema genera el reporte ordenado y lo muestra.
Flujos alternativos	2a. Si no hay datos para el período elegido, el sistema lo informa.

3. Diagrama de clases inicial

3.1 Diagrama - link a Draw.io



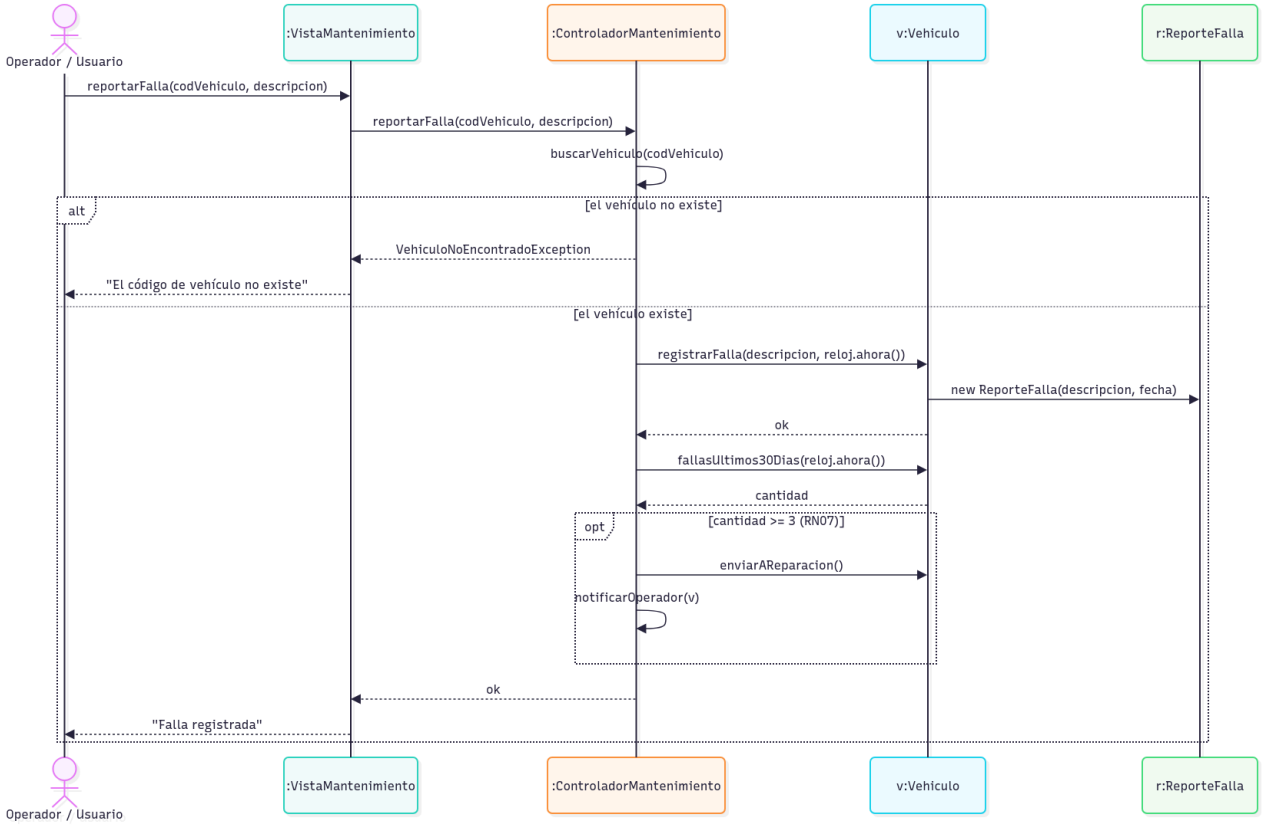
El diagrama inicial representa el dominio tal como surge del enunciado, antes de aplicar los patrones. Una clase Sistema concentra las colecciones y todas las operaciones; Usuario guarda su saldo y sus movimientos; Viaje calcula su propio costo; y Vehiculo tiene el nivel de batería para todos los tipos.

Incluye dos jerarquías de herencia con sobreescritura de métodos: Vehiculo (abstracta) con BicicletaComun, BicicletaElectrica y MonopatinElectrico; y Personal (abstracta) con Operador, Tecnico y Administrador. Las composiciones representan partes que no existen sin su todo (los puntos del recorrido y las notas de ajuste de un viaje, los movimientos de un usuario, los reportes de falla de un vehículo), y las agregaciones representan colecciones que comparten objetos (los vehículos de una estación).

4. Diagramas de secuencia

Se eligieron los casos de uso CU13 Reportar falla y CU15 Registrar traslado, porque ambos tienen flujos alternativos y de error, y muestran la colaboración entre varias clases del dominio (vehículos, reportes de falla, estaciones y traslados).

4.1 CU13 Reportar falla - Link a Draw.io



El usuario o el operador informa el vehículo y la descripción de la falla, y la vista delega en ControladorMantenimiento. Un fragmento alt separa el flujo de error, en el que el código no existe (VehiculoNoEncontradoException), del flujo normal. En el flujo normal, el vehículo registra la falla y crea el ReporteFalla (Creador), y luego el controlador le pide la cantidad de fallas de los últimos 30 días, que el propio vehículo calcula (Experto). Un fragmento opt cubre la RN07: si son 3 o más, el vehículo pasa a "en reparación" y se avisa al operador para que genere la orden de mantenimiento.

4.2 CU15 Registrar traslado - Link a Draw.io



El operador indica la estación de origen, la de destino y los vehículos a mover, y la vista delega en ControladorEstaciones. Un primer fragmento loop recorre los códigos y pide cada vehículo a la estación de origen; un fragmento break corta el traslado con VehiculoNoDisponibleException si alguno no está en la estación o está en uso. Un segundo loop consulta a la estación de destino los lugares libres por tipo, y un fragmento alt lanza EstacionLlenaException si no alcanzan. En el flujo normal, un tercer loop saca cada vehículo del origen y lo entrega al destino, se crea el Traslado con la fecha y el operador responsable, y un fragmento opt genera la alerta de redistribución si alguna de las dos estaciones queda fuera del rango de ocupación (RN11).

5. Aplicación de patrones

5.1 Principios SOLID

Responsabilidad única (SRP)

En el diseño inicial, Usuario guardaba sus datos personales y además manejaba el saldo y los movimientos, Viaje registraba el recorrido y también calculaba su costo, y Sistema hacía todo, por lo que cada una de esas clases tenía más de una razón para cambiar. Para resolverlo se separaron las responsabilidades: la clase Cuenta pasó a manejar el saldo y los movimientos, CalculadorTarifa a calcular el importe de los viajes y GeneradorReportes a armar los reportes. El principio se ve reflejado en Usuario, Cuenta, Movimiento, Viaje, CalculadorTarifa y GeneradorReportes. La ventaja es que un cambio en las tarifas o en las promociones ya no afecta a Viaje ni a Usuario, y que las clases quedan más chicas y cada una se puede probar por separado.

Abierto/Cerrado (OCP)

En el diseño inicial, Promocion tenía un atributo tipo y el cálculo del costo usaba un if por cada tipo de promoción, por lo que agregar una promoción nueva obligaba a modificar ese método. Para resolverlo, Promocion pasó a ser una interfaz con los métodos aplica() y calcularDescuento(), implementada por PromocionPrimerViaje y PromocionPlanMensual, y CalculadorTarifa recorre la lista de promociones y aplica la de mayor descuento (RN18) sin saber de qué tipo es cada una. El principio se ve reflejado en Promocion, PromocionPrimerViaje, PromocionPlanMensual y CalculadorTarifa. La ventaja es que una promoción nueva, por ejemplo un descuento los domingos, se agrega creando una clase nueva, sin modificar el código que ya funciona.

Segregación de interfaces (ISP)

En el diseño inicial, Vehiculo declaraba el atributo nivelBateria y el método getNivelBateria() para los tres tipos. Así, las clases que solo trabajan con baterías, como el registro de carga del CU19 o el control de batería baja al devolver, dependían de toda la interfaz general de Vehiculo, y BicicletaComun quedaba obligada a implementar un método que no usa. Para resolverlo se creó la interfaz Electrico, con getNivelBateria() y actualizarBateria(), implementada solo por BicicletaElectrica y MonopatinElectrico, y Vehiculo quedó solo con lo común a los tres tipos. El principio se ve reflejado en Vehiculo, Electrico, BicicletaComun, BicicletaElectrica, MonopatinElectrico y ControladorMantenimiento. La ventaja es que los clientes que operan con baterías, como ControladorMantenimiento en el CU19, dependen solo de la interfaz chica y específica Electrico y no de los métodos generales de Vehiculo, y que ninguna subclase queda obligada a implementar métodos que no le corresponden.

5.2 Patrones GRASP

Experto en información

En el diseño inicial, Sistema calculaba la ocupación de las estaciones recorriendo sus listas de vehículos y verificaba el saldo leyendo atributos de Usuario. Para resolverlo, cada responsabilidad se asignó a la clase que tiene los datos: Estacion calcula getOcupacion(), estaLlenaPara(tipo) y necesitaRedistribucion(); Cuenta responde tieneSaldo(monto); Vehiculo cuenta sus fallasUltimos30Dias() y crea sus propios reportes de falla; y Viaje calcula getMinutosCobrables(). El patrón se ve reflejado en Estacion, Cuenta, Vehiculo y Viaje. La ventaja es que la lógica queda junto a los datos que usa y no hace falta exponer las colecciones internas, lo que mejora el encapsulamiento.

Creador

En el diseño inicial no estaba definido quién creaba los puntos del recorrido, los movimientos y los viajes, y todo terminaba en Sistema. Para resolverlo se decidió que Viaje cree sus PuntoRecorrido en agregarPunto(), porque los contiene por composición; que Cuenta cree los Movimiento en recargar() y debitar(), porque es la que los registra; que Vehiculo cree sus ReporteFalla en registrarFalla(), porque los contiene por composición; y que ControladorViajes cree los Viaje, porque los registra y tiene todos los datos para inicializarlos. El patrón se ve reflejado en Viaje, PuntoRecorrido, Cuenta, Movimiento, Vehiculo, ReporteFalla y ControladorViajes. La ventaja es que ninguna otra clase necesita conocer cómo se construyen esos objetos, lo que reduce el acoplamiento.

Controlador

En el diseño inicial, Sistema era una clase que recibía todos los eventos y hacía todo el trabajo, y la interfaz gráfica dependía directamente de ella. Para resolverlo se crearon controladores por grupo de casos de uso: ControladorUsuarios (CU01 a CU05), ControladorViajes (CU06 a CU12, CU16 y CU17), ControladorMantenimiento (CU13, CU14, CU18 y CU19), ControladorEstaciones (CU15 y CU20) y ControladorAdministracion (CU21, CU22 y CU23), que concentra las altas de vehículos, personal, tarifas y promociones, mientras que los reportes (CU24) quedaron en GeneradorReportes. El patrón se ve reflejado en ControladorUsuarios, ControladorViajes, ControladorMantenimiento, ControladorEstaciones, ControladorAdministracion y GeneradorReportes. La ventaja es que la interfaz Swing solo captura los eventos y delega, que la lógica de negocio se puede probar sin la interfaz y que cada controlador mantiene alta cohesión.

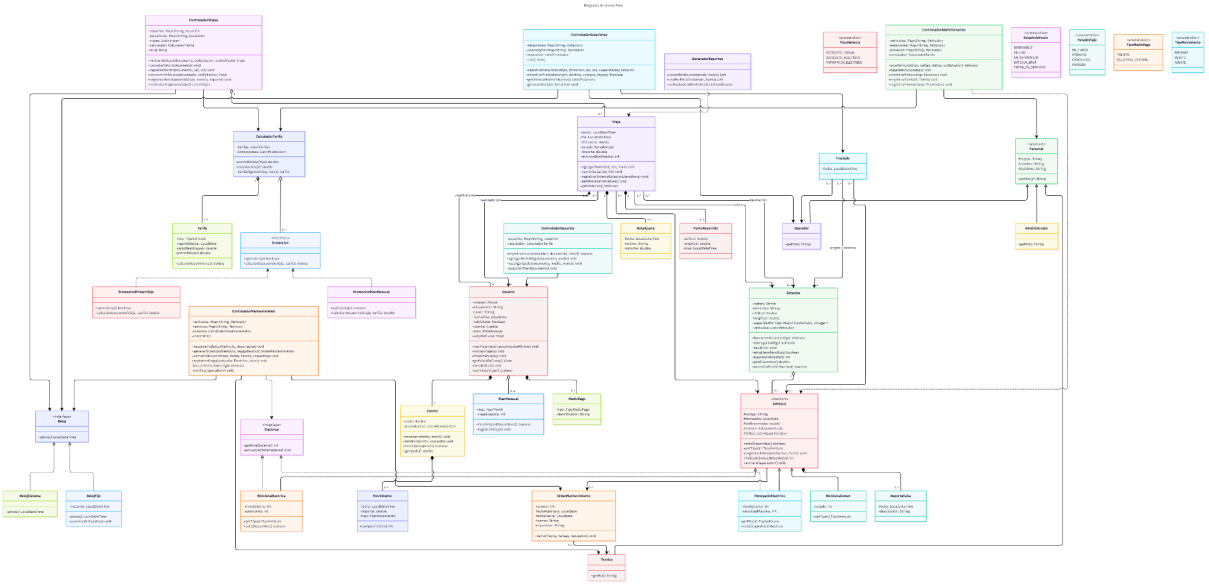
Polimorfismo

La disponibilidad de un vehículo depende de su tipo, ya que los eléctricos además necesitan al menos un 15 % de batería, y el descuento depende del tipo de promoción, por lo que resolverlo con un if por tipo obligaría a modificar el código cada vez que aparece un tipo nuevo. Para resolverlo, Vehiculo es abstracta, con getTipo() abstracto y estaDisponible() sobrescrito en BicicletaElectrica y MonopatinElectrico, y las promociones se resuelven con la interfaz Promocion. Así, quien llama no pregunta el tipo: envía el mensaje y cada objeto responde a su manera. El patrón se ve reflejado en Vehiculo, BicicletaComun,

BicicletaElectrica, MonopatinElectrico, Promocion y sus implementaciones. La ventaja es que un tipo de vehículo nuevo se agrega con una subclase, sin tocar ControladorViajes ni Estacion.

6. Diagrama de clases final

6.1 Diagrama - link a Draw.io



6.2 Cambios respecto del diagrama inicial

1. Sistema se reemplazó por ControladorUsuarios, ControladorViajes, ControladorMantenimiento, ControladorEstaciones, ControladorAdministracion y GeneradorReportes, para repartir los casos de uso y evitar una clase que lo hace todo (Controlador, SRP).

2. El saldo y los movimientos pasaron de Usuario a la nueva clase Cuenta, que además crea los movimientos (SRP, Creador, Experto).

3. El método calcularCosto() salió de Viaje y pasó a la nueva clase CalculadorTarifa, que conoce las tarifas y las promociones (SRP).

4. Promocion dejó de ser una clase con un atributo tipo y pasó a ser una interfaz con PromocionPrimerViaje y PromocionPlanMensual (OCP, Polimorfismo).

5. El nivel de batería salió de Vehiculo y pasó a la interfaz Electrico, implementada solo por los vehículos eléctricos, que además sobrescriben estaDisponible() (ISP, Polimorfismo).

6. Estacion incorporó buscarVehiculo(), entregar(), recibir(), estaLlenaPara(), lugaresLibres(), getOcupacion() y necesitaRedistribucion(), que antes estaban en Sistema (Experto).

7. Viaje incorporó agregarPunto(), cerrar(), registrarIntentoEstacionLlena() y getMinutosCobrables() (Experto, Creador).

8. Usuario incorporó la referencia a su viaje en curso y los métodos verificarHabilitacion(), iniciarViaje(), finalizarViaje() e inhabilitar(), para validar RN01 a RN03.

9. Se agregaron PlanMensual, para contar los viajes incluidos en el mes, y la interfaz Reloj (S21), implementada por RelojSistema, que da la hora real, y RelojFijo, que permite fijar y avanzar el tiempo en las pruebas. Los controladores que registran fechas (ControladorViajes, ControladorMantenimiento y ControladorEstaciones) usan un Reloj.

10. Movimiento implementa Comparable para ordenar los movimientos por fecha en CU04, y GeneradorReportes usa Comparator para ordenar los vehículos por cantidad de fallas.

11. Los tipos y estados fijos (TipoVehiculo, EstadoVehiculo, EstadoViaje, TipoMedioPago y TipoMovimiento) dejaron de ser String y pasaron a ser enumerativos (enum).

12. Vehiculo incorporó enviarAReparacion(), que se usa en el CU13 cuando el vehículo acumula 3 fallas en 30 días (RN07). Además, todos los controladores detallan sus atributos y las colecciones que administran.

Excepciones propias previstas para las reglas de negocio: SaldoInsuficienteException, UsuarioInhabilitadoException, ViajeAbiertoException, VehiculoNoDisponibleException, VehiculoNoEncontradoException y EstacionLlenaException, todas subclases de una excepción común ReglaNegocioException.